JAVA · CONCURRENCY · INTERVIEW NOTES

Multithreading Important Concepts

9 concepts · one page
core threads → locks → executors → async

1 Create a Thread

- **Extend Thread** and override `run()`. Simple, but the class can no longer extend anything else.
- **Implement Runnable** and pass it to `new Thread(task)`. The task stays separate from the worker.

```
class Worker extends Thread {
  public void run() { work(); }
}
new Worker().start();

Runnable task = () -> work();
new Thread(task, "worker-1").start();
```

The class is the work. The thread is only the worker that runs it.

Prefer Runnable. It keeps your one inheritance slot free and is what executors accept.

2 run() vs start()

- `start()` asks the JVM for a **new thread**, which then calls `run()`.
- `run()` is an ordinary method call on the **current** thread.
- Calling `start()` twice throws `IllegalThreadStateException`.

```
Thread t = new Thread(() ->
  System.out.println(
    Thread.currentThread().getName()));

t.run();    // prints "main"
t.start();  // prints "Thread-0"
```

After TERMINATED, that same Thread object cannot be started again.

Current thread: **Thread.currentThread()** — name, id, and interrupt flag.

3 synchronized · Intrinsic Lock

- Every object has one **monitor**. One thread holds it; others wait. It is reentrant and released automatically, even if the method throws.

```
// method  → lock is this
synchronized void inc() { count++; }

// block   → lock is the object you pass
synchronized (lock) { count++; }

// static  → lock is Counter.class
static synchronized void incTotal() {
  total++;
}
```

On a public class, lock a private final object rather than this.

A static lock and an instance lock are two different locks. They do not exclude each other.

4 sleep() · join() · wait()

CALL	DEFINED ON	LOCK	EFFECT
`sleep(ms)`	Thread, static	Keeps it	Pauses the current thread
`join()`	Thread	Keeps it	Waits until that thread ends
`wait()`	Object	Releases it	Waits for notify or a timeout

```
worker.start();
worker.join();      // caller blocks here
Thread.sleep(500);  // caller pauses 0.5s
```

yield() is only a scheduling hint. Like sleep() and join(), it keeps any lock the thread holds.

All three throw InterruptedException. Only wait() must be called inside synchronized on the same object.

5 wait() · notify() · notifyAll()

```
synchronized void put(T x) throws Exception {
  while (full()) wait();   // releases the lock
  queue.add(x);
  notifyAll();
}
synchronized T take() throws Exception {
  while (empty()) wait();
  T x = queue.remove();
  notifyAll();
  return x;
}
```

- `notify()` wakes one waiter. `notifyAll()` wakes every waiter; they re-check the condition.
- Outside `synchronized`, these throw `IllegalMonitorStateException`.

Prefer notifyAll when several conditions share one lock, so the right waiter is not skipped.

Wait inside a while, not an if. Wake-ups can be spurious, and the condition may already be false again.

6 Daemon Thread · interrupt()

- A **daemon** is a background helper. The JVM exits once only daemon threads remain.
- `setDaemon(true)` must run **before** `start()`.
- `interrupt()` sets a flag. It does not kill the thread. `sleep`, `wait`, and `join` throw `InterruptedException` and clear the flag.

```
while (!Thread.currentThread().isInterrupted()) {
  try { Thread.sleep(500); }
  catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // restore
  }
}
```

A user thread keeps the JVM alive. A daemon thread does not.

Restore the interrupt flag in the catch, or the loop will not see it and will never stop.

7 Runnable vs Callable

TYPE	METHOD	RETURNS	CHECKED EXCEPTIONS
`Runnable`	void run()	nothing	not allowed
`Callable<V>`	V call()	a value	allowed

```
Callable<Integer> task = () -> {
  Thread.sleep(100);   // legal here
  return 42;
};
Future<Integer> f = pool.submit(task);
f.get();               // 42, blocks until done
```

Neither starts work on its own. Something still has to call start() or submit().

Choose Callable when the caller needs a result or a checked exception back.

8 execute() vs submit()

```
ExecutorService pool =
    Executors.newFixedThreadPool(4);

pool.execute(task);          // returns void
Future<?> a = pool.submit(task);       // get() → null
Future<Integer> b = pool.submit(call); // get() → 42
pool.shutdown();
```

- A **Future** is a handle: `get()`, `get(timeout)`, `isDone()`, `cancel()`.
- `submit(Runnable)` exists so you can wait, cancel, and see failures. `get()` wraps them in `ExecutionException`. `execute()` sends them to the uncaught-exception handler.

invokeAll waits for the whole batch. invokeAny returns the first result that succeeds.

Shut the pool down. A non-daemon pool keeps the JVM alive.

9 CompletableFuture

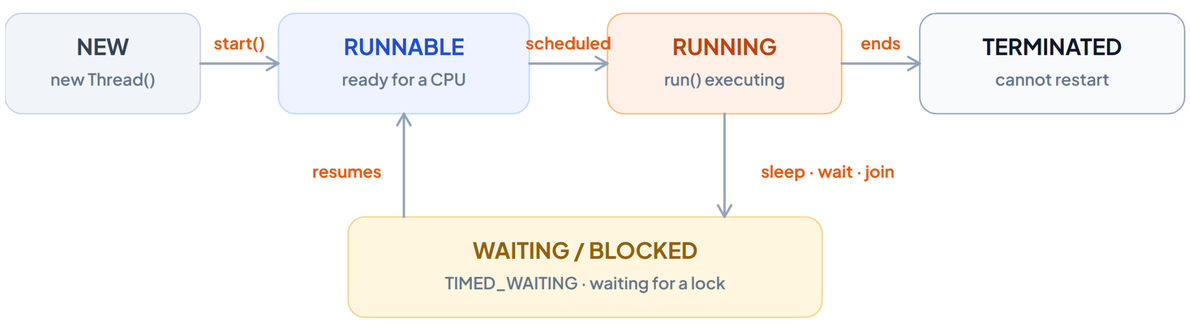
- `runAsync` takes a Runnable and returns `CompletableFuture<Void>`.
- `supplyAsync` takes a Supplier and returns `CompletableFuture<T>`.
- Default executor: `ForkJoinPool.commonPool()`. Pass your own for blocking I/O.

```
CompletableFuture.supplyAsync(() -> 10)
    .thenApply(x -> x * 2)
    .thenAccept(System.out::println)
    .join();                 // prints 20
```

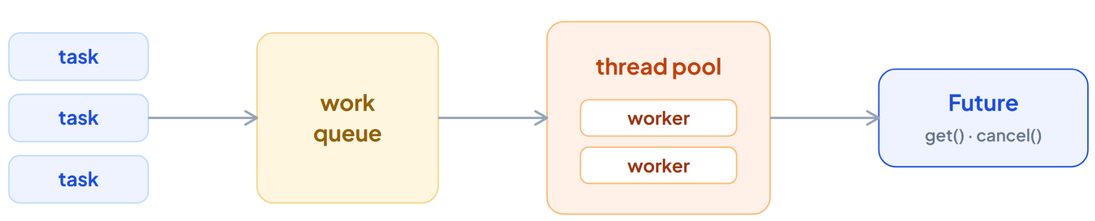
thenApply maps a value. thenCompose flattens the next future. thenAccept consumes it.

A Future only blocks on get(). A CompletableFuture chains, combines, and handles errors without blocking. join() is unchecked; get() is checked.

Thread execution



ExecutorService



submit() queues the task · workers are reused · shutdown() when finished